

DAY-14

JAVA CONCEPTS

1. Serialization:

Serialization is the process of converting a Java object into a byte stream so that it can be stored in a file or transferred over a network.

Why it is used

- Save object data permanently.
- Send objects over a network.
- Store application state.

Real-time Example

Saving a student's details into a file and retrieving them later.

Java Program

```
import java.io.*;

class Student implements Serializable {

    int id;

    String name;

    Student(int id, String name) {

        this.id = id;

        this.name = name;

    }

}

public class SerializationDemo {

    public static void main(String[] args) {

        try {

            Student s = new Student(101, "Sindhu");

            FileOutputStream fos = new FileOutputStream("student.ser");

            ObjectOutputStream oos = new ObjectOutputStream(fos);

            oos.writeObject(s);

            oos.close();

        } catch (IOException e) {

            e.printStackTrace();

        }

    }

}
```

```
fos.close();

System.out.println("Object Serialized Successfully");

    }
catch(Exception e) {
    System.out.println(e);
    }
}
}
```

Output

Object Serialized Successfully

Explanation

- Serializable interface enables serialization.
 - ObjectOutputStream writes the object into a file.
 - student.ser stores the object.
-

2. Exception Handling:

Exception Handling is a mechanism to handle runtime errors without terminating the program.

Why it is used

- Prevents crashes.
- Maintains normal program execution.
- Improves reliability.

Real-time Example

ATM withdrawal with insufficient balance.

Basic Flow

Try: Write code that may cause an exception.

Catch/Except: Handle the exception if it occurs.

Finally: Execute cleanup code regardless of whether an exception occurs.

Throw/Raise (optional): Create and trigger an exception.

Types of Exceptions (Java):

Checked Exceptions: Checked at compile time (e.g., IOException, SQLException).

Unchecked Exceptions: Occur at runtime (e.g., ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException).

Java Program

```
public class ExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            System.out.println(a / b);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        }  
        System.out.println("Program Continues...");  
    }  
}
```

Output

Cannot divide by zero.

Program Continues...

Explanation

- Risky code is inside try.
 - Exception is handled in catch.
 - Program continues after handling.
-

3. Java Collections:

Collections Framework provides classes to store and manipulate groups of objects.

Why it is used

- Dynamic storage.
- Easy searching and sorting.
- Better memory management.

Real-time Example

Managing student names.

Main Interfaces

Collection

|

└─ List

| └─ ArrayList

| └─ LinkedList

| └─ Vector

|

└─ Set

| └─ HashSet

| └─ LinkedHashSet

| └─ TreeSet

|

Queue

└─ PriorityQueue

└─ LinkedList

Java Program

```
import java.util.*;

public class CollectionDemo {

    public static void main(String[] args) {
```

```
ArrayList<String> list = new ArrayList<>();  
list.add("Sindhu");  
list.add("Rahul");  
list.add("Anu");  
for(String name : list){  
System.out.println(name);  
    }  
}  
}
```

Output

Sindhu

Rahul

Anu

Explanation

- ArrayList stores elements dynamically.
 - add() inserts elements.
 - Enhanced for loop displays elements.
-

4. Priority Queue:

Priority Queue stores elements according to priority.

Why it is used

- Fast retrieval.
- Scheduling tasks.
- Priority-based processing.

Real-time Example

Hospital emergency patients.

Java Program

```
import java.util.*;
```

```
public class PriorityQueueDemo {  
    public static void main(String[] args) {  
        PriorityQueue<Integer> pq = new PriorityQueue<>();  
        pq.add(50);  
        pq.add(20);  
        pq.add(10);  
        pq.add(30);  
        while(!pq.isEmpty()){  
            System.out.println(pq.poll());  
        }  
    }  
}
```

Output

10
20
30
50

Explanation

- Elements are automatically sorted.
- poll() removes the smallest element first.

5. File Handling:

File Handling allows Java programs to create, read and write files.

Why it is used

- Store permanent data.
- Read existing files.
- Generate reports.

Real-time Example

Saving employee details into a text file.

Java Program

```
import java.io.*;

public class FileDemo {

    public static void main(String[] args) {
        try {

            FileWriter fw = new FileWriter("demo.txt");

            fw.write("Welcome to Java");

            fw.close();

            FileReader fr = new FileReader("demo.txt");

            int ch;

            while((ch = fr.read()) != -1){

                System.out.print((char)ch);

            }

            fr.close();

        } catch(Exception e){

            System.out.println(e);

        }

    }

}
```

Output

Welcome to Java.

Explanation

- FileWriter writes data.
- FileReader reads data.
- read() returns one character at a time.

Operations in File Handling

- Java provides different operations on files:

1. Create a File

- Creates a new file.
- Example:
- `File file = new File("demo.txt");`
- `file.createNewFile();`

2. Write to a File

- Writes data into a file.
- Example:
- `FileWriter fw = new FileWriter("demo.txt");`
- `fw.write("Hello Java");`
- `fw.close();`

3. Read a File

- Reads data from a file.
- Example:
- `FileReader fr = new FileReader("demo.txt");`

4. Delete a File

- Deletes a file.
- Example:
- `file.delete();`

Classes Used in Java File Handling:

1. File Class

Definition: Used to create or represent a file.

Code

```
import java.io.File;

public class FileExample {
    public static void main(String[] args) throws Exception
    {

        File file = new File("demo.txt");

        if (file.createNewFile()) {
            System.out.println("File Created Successfully");
        } else {
            System.out.println("File Already Exists");
        }
    }
}
```

Output : (First Time)

File Created Successfully

2. FileWriter

Definition: Used to write data into a file.

Code

```
import java.io.FileWriter;

public class FileWriterExample {
    public static void main(String[] args) throws Exception
    {

        FileWriter fw = new FileWriter("demo.txt");

        fw.write("Hello Java");

        fw.close();

        System.out.println("Data Written Successfully");
    }
}
```

Output

Data Written Successfully

3. FileReader

Definition: Used to read data from a file.

Code

```
import java.io.FileReader;

public class FileReaderExample {
    public static void main(String[] args) throws Exception
    {

        FileReader fr = new FileReader("demo.txt");

        int ch;

        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }

        fr.close();
    }
}
```

Output

Hello Java

4. BufferedWriter:

Used to write data efficiently using a buffer.

Code

```
import java.io.BufferedWriter;
import java.io.FileWriter;

public class BufferedWriterExample {
```

```
        public static void main(String[] args) throws Exception
        {

            BufferedWriter bw = new BufferedWriter(new
            FileWriter("demo.txt"));

            bw.write("Welcome to Java");

            bw.close();

            System.out.println("Data Written");

        }
    }
}
```

Output

Data Written

5. BufferedReader

Definition: Used to read data efficiently line by line.

Code

```
import java.io.BufferedReader;
import java.io.FileReader;

public class BufferedReaderExample {
    public static void main(String[] args) throws Exception
    {

        BufferedReader br = new BufferedReader(new
        FileReader("demo.txt"));

        String line = br.readLine();

        System.out.println(line);

        br.close();

    }
}
```

Output

Welcome to Java

6. Scanner

Definition: Used to read data from a file easily.

Code

```
import java.io.File;
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) throws Exception
    {

        File file = new File("demo.txt");
```

```
        Scanner sc = new Scanner(file);

        while (sc.hasNextLine()) {
            System.out.println(sc.nextLine());
        }

        sc.close();
    }
}

Output
Welcome to Java
```

6. Annotations:

Annotations provide additional information to the compiler or JVM.

Why it is used

- Improves readability.
- Detects errors.
- Reduces coding effort.

Real-time Example

Method overriding using @Override.

Java Program

```
class Parent{

    void display(){

        System.out.println("Parent");

    }

}

class Child extends Parent{

    @Override

    void display(){

        System.out.println("Child");

    }

}

public class AnnotationDemo{
```

```
public static void main(String[] args){  
    Child c = new Child();  
    c.display();  
    }  
}
```

Output

Child

Explanation

- @Override verifies method overriding.
- Compiler checks for mistakes.

Common Annotations in Java:

1. @Override

- Indicates that a method overrides a parent class method.
- Helps the compiler detect mistakes.

@Override

```
public void display() {  
    System.out.println("Display");  
}
```

2. @Deprecated

- Marks a method or class as **outdated**.
- The compiler shows a warning when it is used.

@Deprecated

```
public void oldMethod() {  
}
```

3. @SuppressWarnings

- Suppresses compiler warnings.

@SuppressWarnings("unchecked")

4. @FunctionalInterface

- Ensures an interface has **only one abstract method**.
- Used with **Lambda Expressions**.

@FunctionalInterface

```
interface Demo {  
    void show();  
}
```

5. @SafeVarargs

- Suppresses warnings related to **varargs** with generic methods or constructors.

@SafeVarargs

```
public static void print(String... names) {  
}
```

7. Database Connection (JDBC):

JDBC is an API used to connect Java applications with databases.

Why it is used

- Insert records.
- Update records.
- Delete records.
- Retrieve records.

Real-time Example

Employee Management System.

Java Program

```
import java.sql.*;  
  
public class JdbcDemo {  
    public static void main(String[] args) {  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/test",
    "root",
    "password");
System.out.println("Database Connected Successfully");
con.close();
} catch (Exception e){
    System.out.println(e);
}
}
}
```

Output

Database Connected Successfully

Explanation

- Loads MySQL driver.
- Creates connection.
- Prints success message.
- Closes connection.

8. Design Pattern (Singleton):

Design Patterns are reusable solutions for common software design problems.

Why it is used

- Reusable code.
- Better design.
- Easy maintenance.

Real-time Example

Printer spooler or database connection manager where only one instance should exist.

Java Program

```
class Singleton {  
    private static Singleton obj = new Singleton();  
    private Singleton(){}  
    public static Singleton getInstance(){  
        return obj;  
    }  
    public void display(){  
        System.out.println("Singleton Object Created");  
    }  
}  
  
public class SingletonDemo {  
    public static void main(String[] args){  
        Singleton s = Singleton.getInstance();  
        s.display();  
    }  
}
```

Output

Singleton Object Created

Explanation

- Constructor is private, so no other class can create an object.
- `getInstance()` returns the same object every time.
- Ensures only one instance exists throughout the application.

9. Multi-Threading:

Multithreading is a process of executing two or more threads simultaneously to perform multiple tasks at the same time.

Why it is Used

- Improves program performance.
- Executes multiple tasks simultaneously.
- Saves CPU time.
- Makes applications more responsive.

Real-Time Example

Downloading a file while listening to music on your computer.

Java Program

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();

        System.out.println("Main thread is running...");
    }
}
```

Output

Main thread is running...

Thread is running...

Note: Sometimes the output may also appear as:

Thread is running...

Main thread is running...

This is normal because both threads run independently.

Explanation

- class MyThread extends Thread creates a new thread.
 - run() contains the task performed by the thread.
 - start() starts the new thread.
 - The main() method is also a thread (called the main thread).
 - Both threads execute simultaneously.
-

10. Lambda Expressions

A **Lambda Expression** is a short way of writing an anonymous (nameless) function in Java. It is introduced in **Java 8** to make the code shorter and more readable.

Why is it Used?

- Reduces the amount of code.
- Makes the program easy to read.
- Used with functional interfaces.
- Improves code readability.

Real-Time Example

Sorting a list of student names or performing simple calculations using less code.

Syntax

(parameters) -> expression

or

(parameters) -> {

 // statements

}

Simple Java Program

```
interface Message {
```

```
    void display();
```

```
}
```

```
public class LambdaDemo {
```

```

public static void main(String[] args) {

    Message msg = () -> System.out.println("Hello from Lambda Expression");
    msg.display();

}
}

```

Output

Hello from Lambda Expression

Explanation

- Message is a **functional interface** because it has only one abstract method.
 - () -> is the **Lambda Expression**.
 - It provides the implementation for the display() method.
 - msg.display() executes the lambda expression and prints the message.
-

Difference Between throw and throws in Java

throw	throws
throw is a keyword used to explicitly throw an exception .	throws is a keyword used to declare that a method may throw one or more exceptions .
Used inside a method .	Used in the method declaration .
Throws one exception object at a time.	Can declare multiple exception classes .
Followed by an exception object .	Followed by exception class names .

Definition of throw:

throw is a keyword used to **explicitly create and throw an exception** from within a method or block of code.

Example:

```
throw new ArithmeticException("Division by zero");
```

Definition of throws:

throws is a keyword used in the **method declaration** to indicate that the method **may throw one or more exceptions**, allowing the caller to handle them.

Example:

```
void readFile() throws IOException {  
    // code  
}
```

Difference Between ArrayList and LinkedList (Java)

Definition of ArrayList

ArrayList is a class in the Java Collection Framework that **stores elements in a dynamic array**. It provides **fast access (retrieval)** to elements using indexes.

Definition of LinkedList

LinkedList is a class in the Java Collection Framework that **stores elements as nodes connected by links (pointers)**. It provides **fast insertion and deletion** of elements.

ArrayList vs LinkedList

ArrayList	LinkedList
Uses a dynamic array internally.	Uses a doubly linked list internally.
Fast for accessing elements (get()).	Slower for accessing elements because it traverses nodes.
Insertion and deletion are slower, especially in the middle, because elements are shifted.	Insertion and deletion are faster because only links are updated.
Uses less memory.	Uses more memory due to storing node links.
Best for read/search operations.	Best for frequent insert and delete operations.

Example: ArrayList

```
import java.util.*;
```

```
ArrayList<String> list = new ArrayList<>();  
list.add("Java");  
list.add("Python");  
System.out.println(list);
```

Example: LinkedList

```
import java.util.*;  
  
LinkedList<String> list = new LinkedList<>();  
list.add("Java");  
list.add("Python");  
System.out.println(list);
```

JSON (JavaScript Object Notation):

JSON is a lightweight text-based format used to **store and exchange data** between applications, especially between a client (frontend) and a server (backend).

JSON Syntax

- Data is stored as **key-value pairs**.
- Keys and string values are enclosed in **double quotes**.
- Objects use { }.
- Arrays use [].

Example JSON

```
{  
  "id": 101,  
  "name": "Sindhu",  
  "course": "Java Full Stack",  
  "skills": ["Java", "HTML", "CSS"]  
}
```

Real-life Scenario

Imagine you fill out an online registration form.

- **Name:** Sindhu
- **Age:** 21
- **Course:** Java Full Stack

The frontend sends this data to the backend in JSON format:

```
{  
  "name": "Sindhu",  
  "age": 21,  
  "course": "Java Full Stack"  
}
```

The backend reads this JSON, processes it, and stores it in the database.

Difference Between final, finally, and finalize() in Java

1. final –

final is a **keyword** used to restrict changes in Java.

Uses:

- A **final variable** cannot be changed (constant).
- A **final method** cannot be overridden.
- A **final class** cannot be inherited.

Example:

```
final int age = 25;  
  
// age = 30; // Error
```

2. finally – Definition

finally is a **block** used in exception handling. It contains code that is executed **always**, whether an exception occurs or not.

Example:

```
try{  
    int a = 10 / 2;
```

```

}
catch(Exception e) {
    System.out.println(e);
}
finally {
    System.out.println("Cleanup code executed");
}

```

Use: Closing files, database connections, releasing resources.

3. finalize() – Definition

finalize() is a **method** of the Object class that was used by the Garbage Collector before removing an object from memory.

Example:

```

class Demo {
    protected void finalize() {
        System.out.println("Object destroyed");
    }
}

```

Note: finalize() is deprecated in modern Java and is not recommended for resource cleanup.

Difference

final	finally	finalize()
It is a keyword .	It is a block .	It is a method .
Used for restricting variables, methods, and classes.	Used with try-catch for exception handling.	Used for garbage collection cleanup.
Executed at compile time.	Executed after try/catch execution.	Called by Garbage Collector (if applicable).

final

Cannot be
changed/overridden/inherited.

finally

Always executes (except
in rare cases like JVM
shutdown).

finalize()

Deprecated in Java.